Passioné — Full Project Documentation

This document explains everything in the project: the problem it solves, how the system is structured, what every file and module does, how the pieces talk to each other, and what design decisions were made and why.

---

1. What the project is

Passioné predicts whether a newly uploaded movie or short film is likely to trend, based on its metadata — genre, duration, upload time, language, and tags. A person submits that metadata and receives a trending-likelihood score, backed by a classifier that was trained on historical content and evaluated honestly (precision, recall, F1, ROC-AUC — not just a raw "yes/no" guess).

The system is built as three independently owned parts that communicate over defined interfaces rather than by importing each other's internals:

Part	What it does	Owner
Backend (`backend/`)	A FastAPI service exposing training, status, prediction, health, and metrics endpoints	Backend owner
ML (`ml/`)	A Python package, `mlcore`, that owns feature engineering, model training, and prediction logic	ML owner

Frontend (`frontend/`)	A minimal web UI for submitting metadata and viewing scores	UI owner

---

2. Why the system is split this way

The core design principle is: **the backend never imports code from `ml/` or reasons about how predictions are made — it only calls a fixed, agreed-upon interface and passes along whatever comes back.** This exists so that three people can build in parallel without stepping on each other, and so that the ML side can change its internal approach entirely (swap Logistic Regression for a tree ensemble, add text embeddings, whatever) without the backend or frontend needing a single line changed.

There are two integration seams in the project, and neither is a shared codebase:

Seam 1 — backend ↔ ml. The backend imports exactly one thing from `ml/`: six functions from `mlcore.api`. Nothing else. Those six functions and their input/output shapes are frozen — the ML owner can rewrite everything inside them (the actual training code, the actual model) without breaking the backend, as long as the six signatures stay the same.

Seam 2 — backend ↔ frontend. The frontend never runs backend code or knows anything about `mlcore`. It only knows the HTTP contract documented in `docs/API_CONTRACT.md` — it sends requests, gets JSON back, and renders it.

---

3. The mlcore interface (the backend/ML seam), in full

`mlcore.api` exposes exactly six functions:

`health_check() -> dict`

Reports whether a model is currently loaded and ready. Returns a dict with `status`, `model_loaded`, `model_version`, `metrics_ok`, `roc_auc`, and `detail`. This function is documented as never raising an exception — it always returns a well-formed dict, even if nothing has been trained yet.

`load_model(version: str | None = None) -> dict`

Loads a specific model version, or the latest one if no version is given. Idempotent — calling it repeatedly with no model on disk is safe and just returns `loaded: False`.

`get_model_metadata() -> dict`

Returns everything about the currently loaded model that a UI might want to display: algorithm used, when it was trained, the random seed, the label strategy, feature list, metrics, train/test row counts, positive class rate, whether it passed the quality gate, and the categories it was trained on (used for detecting unseen inputs).

`predict(metadata: dict) -> dict`

The core prediction function. Takes a single content item's metadata (title, genre, duration, upload time, language, tags) and returns a trending probability, a predicted label, a confidence score and band, the model version that produced the prediction, the top contributing features, whether the prediction is `grounded` (i.e., the model was genuinely confident, not guessing), and a plain-language `reason` when it isn't. This function is documented as never raising — bad input results in an ungrounded response, not an error.

`predict_batch(items: list[dict]) -> list[dict]`

Same as `predict`, applied to a list of items. Each item gets its own independent result; one bad item in the batch does not fail the others.

`train_model(csv_path, progress_cb, seed, algo) -> dict`

Trains (or retrains) the model from a CSV file. Accepts a callback (`progress_cb`) that gets called during training with row counts, which is how the backend reports live progress. Returns the final model version, row counts (received/processed/failed), evaluation metrics, whether the new model passed the quality gate, and whether it was promoted to be the active model.

Current state of these six functions: all six are implemented as *dummy* logic right now. They return well-formed, contract-shaped responses — the exact same shapes real implementations will return — but with placeholder values: `grounded: False`, `confidence: 0.5`, empty metrics, zero row counts. This is a deliberate build strategy (build the full pipe end-to-end before the model exists), not an incomplete implementation of the interface itself — the interface is complete and frozen; only what happens *inside* it is a placeholder.

---

4. The backend, module by module

`app/main.py`

The FastAPI application factory. Sets up CORS (allowed origins come from config, not hardcoded), registers all routers, and registers the global error handlers.

`app/api/routes/`

One file per group of endpoints. Each route function does three things and no more: validate the incoming request against a Pydantic schema, call exactly one service function, and return a response schema. Any real logic (deciding what a response means, not just shaping it) lives in `services/`, not here.

- **`health.py`** → `GET /health`
- **`train.py`** → `POST /train`
- **`status.py`** → `GET /status`
- **`predict.py`** → `POST /predict`
- **`metrics.py`** → `GET /metrics`
- **`models.py`** → `GET /models`, `POST /models/activate`

`app/schemas/`

Every request and response the API accepts or returns is a Pydantic model defined here. No raw dictionaries cross the API boundary — this means the API's shape is enforced by the type system, not by convention, and any accidental shape mismatch is caught immediately rather than silently serialized wrong.

`app/services/`

Business logic lives here, separated from the HTTP layer:

- **`job_manager.py`** — an in-memory registry of training jobs. Tracks each job's status (`queued`/`training`/`complete`/`failed`), real row counts as they're processed, and any failure message. This is what `/status` reads from.
- **`training_service.py`** — orchestrates a training run as a FastAPI background task, so `/train` can return immediately (`202 Accepted`) instead of blocking the HTTP request for the duration of training. Wraps the entire training call in error handling so any failure — even an unexpected one — lands the job at `status: "failed"` with a message, rather than leaving `/status` stuck reporting `"training"` forever.

- **`model_registry.py`** — *removed.* This existed in an earlier design where the backend discovered and loaded model files from disk directly. Once the ML integration switched to the `mlcore` import-based interface, this responsibility moved entirely into `mlcore` itself, and the file no longer exists in the backend.
- **`prediction_service.py`** — sits between the `/predict` route and the ML adapter. Owns exactly one decision: whether the service is ready to attempt a prediction at all (based on `MIN_ROC_AUC` and whether a model is loaded). If not ready, it raises before ever calling `mlcore.predict()`, and the route returns `409 MODEL_NOT_READY`. It does **not** re-decide whether an individual prediction is grounded — that judgment belongs entirely to `mlcore`, and is passed through unmodified.
- **`enrichment.py`** — an HTTP client for optionally enriching metadata via the TMDB API (extra genre/language/runtime detail beyond what the user submits). Uses `httpx.AsyncClient` with a timeout and a capped retry count; on any failure (timeout, bad response, network error) it falls back to metadata-only rather than blocking or failing the request. Controlled entirely by the `ENRICHMENT_ENABLED` setting.

`app/ml_adapter/`

- **`mlcore_client.py`** — the *only* file in the backend that imports `mlcore`. It wraps every one of the six `mlcore` calls in `try/except`, so an unexpected exception from `mlcore` never becomes an unhandled 500 — it degrades into the same "not ready / ungrounded" shape a normal response would have. It also maps `mlcore`'s dict keys field-by-field into the backend's own Pydantic schemas, so a naming difference on either side is handled explicitly in one place rather than by accident. It applies `MIN_ROC_AUC` as a readiness gate on top of whatever `mlcore` reports about itself.
- **`interface.py`**, **`joblib_loader.py`**, **`stub.py`** — *removed.* These existed under the earlier file-based (joblib artifact) integration design, before the ML owner delivered the frozen `mlcore` import interface. They no longer exist in the codebase.

`app/core/`

- **`config.py`** — the single place environment variables are read, using `pydantic-settings`. Every configurable value in the system (model quality thresholds, timeouts, CORS origins, TMDB settings) is declared here. No other file reads `os.getenv` directly.
- **`errors.py`** — defines a small hierarchy of application errors (`ModelNotReady`, `JobNotFound`, `EmptyDataset`, `BadCSV`, etc.), each mapping to a specific HTTP status code, and one global exception handler that turns any of them — or any unexpected exception — into the same JSON error envelope, so a client never sees a raw stack trace.
- **`logging.py`** — configures structured, timestamped logging output.

---

5. Every API endpoint, in detail

All requests/responses use the shapes defined in `docs/API_CONTRACT.md`. All errors share one envelope: `{"error": {"code": ..., "message": ..., "detail": ...}}`.

`GET /health`

Reports service readiness. `status` is `"ok"` only when a model is loaded *and* its ROC-AUC clears `MIN_ROC_AUC` (currently `0.60`). Otherwise `"degraded"`. This distinction matters because a container that has started is not the same thing as a service that is safe to serve predictions from — a model can finish training and still fail this gate if its quality isn't good enough.

`POST /train`

Accepts a CSV file as `multipart/form-data` (field name `file`), with an optional `random_seed` form field (default `42`). Returns `202`

`Accepted` immediately with a `job_id`, and training runs in the background. This is the only working ingestion mode — an earlier design also allowed submitting a JSON body with a server-local file path, but this was found not to work (FastAPI was silently binding those fields as query parameters instead) and was removed rather than fixed, since no real client would use that mode anyway.

`GET /status?job_id=...`

Reports the real-time state of a training job: `queued`, `training`, `complete`, or `failed`, along with actual row counts (`rows_total`, `rows_processed`, `rows_failed`). A job only reports `complete` once `rows_processed + rows_failed` equals `rows_total` — partial progress is reported honestly while a job is still running, and a genuine failure (including one caused by `mlcore` returning an unexpected shape) always resolves to `failed` with a message, never leaves the job stuck.

`POST /predict`

Accepts a single content item's metadata and returns a trending probability, predicted label, confidence, model version, top contributing features, and whether the prediction is grounded. If the service isn't ready (no model loaded, or the loaded model doesn't clear `MIN_ROC_AUC`), this returns `409 MODEL_NOT_READY` before ever attempting a prediction. If the service *is* ready but the model itself is uncertain about this particular input (e.g. an unseen genre), the response is a normal `200` with `grounded: false` and a `reason` explaining why — that judgment comes entirely from `mlcore`, unmodified by the backend.

`GET /metrics`

Returns the currently active model's evaluation metrics: precision, recall, F1, ROC-AUC, accuracy, positive class rate, the random seed used, the train/val/test split sizes, and when it was trained.

`GET /models`

Reports the currently active model version. Because the frozen `mlcore` interface has no "list all versions" function, this can currently only report one version — it cannot list historical versions for rollback purposes, which is a real, documented limitation of the current interface.

`POST /models/activate`

Requests a specific model version be made active (intended to support rolling back a bad retrain).

---

6. Design decisions and why they were made

Import-based ML integration instead of a file handoff. The original plan had the ML side write a versioned model artifact (`model_vN.joblib`) and a metrics file to disk, which the backend would discover and load. This was replaced with the current design — a frozen Python import (`mlcore.api`) — once the ML owner delivered a working six-function interface. The tradeoff: the backend and `mlcore` now run in the same process and must have compatible dependency versions, whereas the file-based design would have kept them fully isolated. In exchange, integration started immediately with a real, testable interface instead of waiting on a file format to be finalized and built.

mlcore owns prediction semantics; the backend owns readiness only. An earlier version of the prediction adapter independently recomputed whether a prediction was `grounded`, using its own confidence threshold on top of whatever `mlcore` already decided. This meant two different parts of the system could each declare a prediction grounded or not, with no single source of truth. It was changed so that `grounded`, `predicted_label`, and `confidence` pass through from `mlcore` completely unmodified — the backend's only remaining judgment is whether the *service* is ready to take predictions at all (the

`409` gate), which is a deployment concern, not a prediction-quality judgment.

Multipart-only training ingestion. Covered above under `/train` — a second ingestion mode (JSON body with a server-side file path) was designed but found to be nonfunctional and was removed rather than repaired, since it wasn't a mode any real client would actually use.

---

7. Configuration

Every configurable value lives in `backend/app/core/config.py`, sourced from environment variables (with `.env.example` documenting every one, with no real values committed). Notable settings:

- `MIN_ROC_AUC` (`0.60`) — the quality floor a model must clear for `/health` to report `"ok"` and for `/predict` to be attempted at all.
- `ENRICHMENT_ENABLED` (`false`) — whether the TMDB enrichment client is used. Currently off; the system runs on submitted metadata only.
- `CORS_ORIGINS` — which frontend origins are allowed to call the API.
- TMDB-related settings (base URL, timeout, retry count) — used only when enrichment is enabled.

`GROUNDING_MIN_CONFIDENCE` existed as a setting earlier in the project but was removed entirely once the grounding-ownership decision above was made — there is no remaining use for a backend-side confidence threshold now that `mlcore` owns that judgment alone.

---

8. Testing

The backend test suite covers:

- **Contract shape** — every response matches what `docs/`

`API_CONTRACT.md` documents.
- **Hostile input** — an empty training CSV, a CSV with a header but no rows, a `/predict` request missing a required field, a `/predict` request with a genre/language never seen in training, and a `/predict` request sent before any model exists. Each returns a clean error, never a stack trace.
- **Reproducibility** — training twice against the same data and seed produces identical metrics.
- **Readiness boundary** — `/health` correctly reports `"ok"` at and above `MIN_ROC_AUC`, and `"degraded"` just below it — confirming the check is a real `>=` comparison, not off-by-one.
- **Partial training failure** — `/status` correctly reports nonzero `rows_failed` alongside `rows_processed` when some rows fail during training, and correctly distinguishes an in-progress job from a completed one.
- **Total training failure** — an unexpected exception during training (including a metrics response shaped differently than expected) results in `/status` reporting `"failed"` with a message, not a stuck job or an unhandled error.
- **Grounding ownership** — confirms `mlcore`'s `grounded`/`predicted_label`/`confidence` values pass through unmodified, and confirms `mlcore.predict()` is never even called when the service isn't ready.

---

9. Current state, plainly

The backend's plumbing — every endpoint, the error handling, the background-job system, the ML integration seam, the readiness gating — is complete and has real test coverage behind each piece described above. What has not yet happened is real model training: `mlcore`'s six functions are still running their dummy implementations, so every prediction the system currently produces is a placeholder response (`grounded: false`, fixed confidence), not a real trending-likelihood assessment. The system is built to receive a real model without any change to its own code; it has not yet received one.